

实验四 垃圾邮件分类器设计与实现

一、实验目的

1. 了解朴素贝叶斯分类器基本原理，掌握朴素贝叶斯、模型分类器构建方法；
2. 编程实现朴素贝叶斯、文本分类模型；
3. 使用朴素贝叶斯、文本分类模型实现分类，可视化文本数据集分类结果。

二、实验原理

使用 Python 读取文本分类数据集信息并对数据进行预处理，随后利用 Python 实现分类器构建，完成数据加载、数据线性划分、数据预测并使用 scikit-learn 实现数据分类。

三、实验内容

本案例使用 Python 逐步实现了文本分类模型。在实现的过程中，借助 scikit-learn 工具对文本分类模型性能进行提升。

1、加载数据集

(1) 加载数据包

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
from sklearn import feature_extraction, model_selection, naive_bayes, metrics, svm
from IPython.display import Image
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline
```

(2) 加载文本数据集并展示前 10 行；

```
data = pd.read_csv(r'C:\Users\86187\Desktop\spam.csv', encoding='latin-1')
```

(3) 判断正常邮件数量和垃圾邮件数量并画图展示；

```
#总体有多少封正常邮件，多少封垃圾邮件
count_Class=pd.value_counts(data["v1"], sort= True)
count_Class.plot(kind= 'bar', color= ["blue", "orange"])
plt.title('Bar chart')
```

(4) 将上面是数据转化为饼图，求二者比例；

```
将上面的结果化成饼图，求出两者的比例
count_Class.plot(kind = 'pie', autopct='%1.0f%%')
plt.title('Pie chart')
plt.ylabel('')
```

(5) 筛选出正常邮件并将邮件进行分词，转化为列表；

```
count1 = Counter(" ".join(data[data['v1']=='ham']['v2']).split()).most_common(20)
#上面这个语句 是 把所有的正常邮件选择出来，将每个词都分开，转化成列表。然后按照词的频数大小排序，选择出现最频繁的二十个词
df1 = pd.DataFrame.from_dict(count1)
df1 = df1.rename(columns={0: "words in non-spam", 1 : "count"})
#上面这个语句是在改第一列和第二列的列名。
count2 = Counter(" ".join(data[data['v1']=='spam']['v2']).split()).most_common(20)
df2 = pd.DataFrame.from_dict(count2)
df2 = df2.rename(columns={0: "words in spam", 1 : "count_"})
```

(6) 以柱状图的形式标出正常邮件里最高词频的 30 个词；

```
df1.plot.bar(legend = False)
y_pos = np.arange(len(df1["words in non-spam"]))
plt.xticks(y_pos, df1["words in non-spam"])
plt.title('More frequent words in non-spam messages')
plt.xlabel('words')
plt.ylabel('number')
plt.show()
```

(7) 同理，列举出垃圾邮件里最高词频的 30 个词；

```
df2.plot.bar(legend = False, color = 'orange')
y_pos = np.arange(len(df2["words in spam"]))
plt.xticks(y_pos, df2["words in spam"])
plt.title('More frequent words in spam messages')
plt.xlabel('words')
plt.ylabel('number')
plt.show()
```

(8) 不计入英语，词袋模型处理

```
f = feature_extraction.text.CountVectorizer(stop_words = 'english')
X = f.fit_transform(data["v2"])
np.shape(X)
```

Out[49]:

(5572, 8404)

2、分类模型设计

(1) 数据标注及数据集拆分;

```
data["v1"] = data["v1"].map({'spam': 1, 'ham': 0})
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, data['v1'], test_size=0.33, random_state=42)
```

(2) 打印输出训练集、测试集数量;

```
[(3733, 8404), (1839, 8404)]
```

(3) 多项式模型的贝叶斯调整平滑参数将每次模型的预测结果评价放入数组中;

```
list_alpha = np.arange(1/100000, 20, 0.11)
score_train = np.zeros(len(list_alpha))
score_test = np.zeros(len(list_alpha))
recall_test = np.zeros(len(list_alpha))
precision_test = np.zeros(len(list_alpha))
count = 0
for alpha in list_alpha:
    bayes = naive_bayes.MultinomialNB(alpha=alpha)
    bayes.fit(X_train, y_train)
    score_train[count] = bayes.score(X_train, y_train)
    score_test[count] = bayes.score(X_test, y_test)
    recall_test[count] = metrics.recall_score(y_test, bayes.predict(X_test))
    precision_test[count] = metrics.precision_score(y_test, bayes.predict(X_test))
    count = count + 1
```

(4) 计算训练准确率、测试准确率、召回率和精确率;

```
matrix = np.matrix(np.c_[list_alpha, score_train, score_test, recall_test, precision_test])
models = pd.DataFrame(data = matrix, columns =
    ['alpha', 'Train Accuracy', 'Test Accuracy', 'Test Recall', 'Test Precision'])
models.head(n=10)
```

(5) 找到训练集预测准确度最高的模型

```
best_index = models['Test Precision'].idxmax()
models.iloc[best_index, :]
```

Out[53]:

```
alpha          15.730010
Train Accuracy    0.979641
Test Accuracy     0.969549
Test Recall       0.777778
Test Precision    1.000000
Name: 143, dtype: float64
```

3、贝叶斯分类

(1) 模型训练分类;

```
best_index = models[models['Test Precision']==1]['Test Accuracy'].idxmax()
bayes = naive_bayes.MultinomialNB(alpha=list_alpha[best_index])
bayes.fit(X_train, y_train)
models.iloc[best_index, :]
```

(2) 绘制混淆矩阵;

```
m_confusion_test = metrics.confusion_matrix(y_test, bayes.predict(X_test))
pd.DataFrame(data = m_confusion_test, columns = ['Predicted 0', 'Predicted 1'],
             index = ['Actual 0', 'Actual 1'])
```

得到:

Out[55]:

	Predicted 0	Predicted 1
Actual 0	1587	0
Actual 1	62	190

四、总结分析

总结人工神经网络感知机模型分类流程;

分析感知机模型分类结果。